Teacher Guide: Conditional Variational Autoencoders

Architecture, Theory, and Exercise Design

ETH – Business & Machine AI, FS26

Course Staff — Internal Document

March 31, 2026

Contents

1 Overview **2**
- 1.1 Prerequisites . . . 2
- 1.2 Estimated Duration . . . 2

2 Theoretical Background **2**
- 2.1 Generative Models and Latent Variables . . . 2
- 2.2 Variational Autoencoders (VAE) . . . 2
  - 2.2.1 The Evidence Lower Bound (ELBO) . . . 2
  - 2.2.2 Closed-Form KL for Gaussians . . . 3
  - 2.2.3 The Reparametrization Trick . . . 3
- 2.3 Conditional Variational Autoencoders (CVAE) . . . 3
  - 2.3.1 Motivation . . . 3
  - 2.3.2 CVAE ELBO . . . 4
  - 2.3.3 How Conditioning Changes the Architecture . . . 4
- 2.4 The β-VAE Framework . . . 5

3 Architecture Details **5**
- 3.1 Overview . . . 5
- 3.2 Encoder Architecture . . . 6
- 3.3 Decoder Architecture . . . 6
- 3.4 Design Decisions: MNIST vs. CelebA . . . 6

4 File Relationships **7**
- 4.1 Why the Model Lives in the Notebook . . . 7
- 4.2 Function Reference . . . 7

5 TODO Solutions and Grading Notes **8**
- 5.1 TODO 1: ConvCVAE Methods . . . 8
- 5.2 TODO 2: Loss Function . . . 8
- 5.3 TODOs 3–5: Open-Ended Exploration . . . 8

6 Expected Student Difficulties **9**

7 Discussion Points for Class **9**

8 References **9**

1 Overview

This document provides an in-depth reference for the Conditional VAE (CVAE) in-class exercise. It covers the full mathematical derivation, architectural design decisions, file relationships, and pedagogical notes.

Exercise Objectives

After completing this exercise, students should be able to:

1. Explain the difference between AE, VAE, and CVAE architectures
2. Implement the reparametrization trick and understand why it is necessary
3. Build a conditional encoder/decoder that accepts auxiliary label information
4. Train a CVAE and interpret the ELBO loss components
5. Use a trained CVAE for conditional generation and latent interpolation
6. Reason about the β trade-off between reconstruction and regularisation

1.1 Prerequisites

Students should have completed the **Autoencoders CX** exercise and be familiar with:

- Encoder-decoder architectures (convolutional)
- PyTorch basics: `nn.Module`, forward pass, training loops
- MSE loss, Adam optimiser, basic training diagnostics

1.2 Estimated Duration

45–60 minutes for the core TODOs (1–3). The full notebook including β experiments takes approximately 75–90 minutes.

2 Theoretical Background

2.1 Generative Models and Latent Variables

Definition 2.1 (Latent Variable Model)**.** *A latent variable model assumes observed data* $\mathbf{x}$ *is generated by first sampling an unobserved (latent) variable* $\mathbf{z}$ *from a prior* $p(\mathbf{z})$*, then generating* $\mathbf{x}$ *from a conditional distribution* $p_\theta(\mathbf{x}|\mathbf{z})$*:*

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x}|\mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$$

The integral is intractable for complex models (neural networks), which motivates variational inference.

2.2 Variational Autoencoders (VAE)

2.2.1 The Evidence Lower Bound (ELBO)

We want to maximise $\log p_\theta(\mathbf{x})$ but cannot compute it directly. Introducing an approximate posterior $q_\phi(\mathbf{z}|\mathbf{x})$ (the encoder):

Theorem 2.2 (ELBO Derivation)**.**

$$\begin{aligned}
\log p_\theta(\mathbf{x}) &= \log \int p_\theta(\mathbf{x}|\mathbf{z})\, p(\mathbf{z})\, d\mathbf{z} && (1)\\
&= \log \int \frac{q_\phi(\mathbf{z}|\mathbf{x})}{q_\phi(\mathbf{z}|\mathbf{x})} p_\theta(\mathbf{x}|\mathbf{z})\, p(\mathbf{z})\, d\mathbf{z} && (2)\\
&\geq \int q_\phi(\mathbf{z}|\mathbf{x}) \log \frac{p_\theta(\mathbf{x}|\mathbf{z})\, p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})}\, d\mathbf{z} \quad \textit{(Jensen's inequality)} && (3)\\
&= \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right]}_{\textit{Reconstruction term}} - \underbrace{D_{\mathrm{KL}}\left(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})\right)}_{\textit{Regularisation term}} && (4)
\end{aligned}$$

This is the ***Evidence Lower Bound (ELBO)****.*

The gap between $\log p_\theta(\mathbf{x})$ and the ELBO is exactly $D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p_\theta(\mathbf{z}|\mathbf{x}))$, which is always non-negative. Maximising the ELBO simultaneously:

- Pushes the decoder to reconstruct well (reconstruction term)
- Pushes the encoder to match the prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ (KL term)

2.2.2 Closed-Form KL for Gaussians

When $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$ and $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$:

KL Divergence (Gaussian)

$$D_{\mathrm{KL}}\left(q_\phi \| p\right) = -\frac{1}{2} \sum_{j=1}^{d} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$$

In practice, the encoder outputs $\log \sigma^2$ (log-variance) for numerical stability:

$$D_{\mathrm{KL}} = -\frac{1}{2} \sum_{j=1}^{d} \left(1 + \log \mathrm{var}_j - \mu_j^2 - e^{\log \mathrm{var}_j}\right)$$

2.2.3 The Reparametrization Trick

Why Reparametrize?

The encoder defines a distribution $q_\phi(\mathbf{z}|\mathbf{x})$, and we need to sample $\mathbf{z}$ from it during training. However, the `sample()` operation has no gradient. The reparametrization trick makes sampling differentiable:

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Now $\mathbf{z}$ is a deterministic function of ϕ (and random ε), so gradients ∇_ϕ flow through $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$.

2.3 Conditional Variational Autoencoders (CVAE)

2.3.1 Motivation

A standard VAE models $p(\mathbf{x})$ — it can generate data but has no control over *what* it generates. A CVAE models $p(\mathbf{x}|\mathbf{c})$ where $\mathbf{c}$ is a condition (e.g., digit label, face attributes):

Definition 2.3 (Conditional VAE)**.** *A CVAE extends the VAE by conditioning all distributions on an observed variable* $\mathbf{c}$*:*

- ***Encoder****:* $q_\phi(\mathbf{z}|\mathbf{x}, \mathbf{c})$ *— infers latent representation given* both *data and condition*
- ***Decoder****:* $p_\theta(\mathbf{x}|\mathbf{z}, \mathbf{c})$ *— generates data given latent code and condition*
- ***Prior****:* $p(\mathbf{z}|\mathbf{c})$ *— prior over latent space (often assumed label-independent:* $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$*)*

2.3.2 CVAE ELBO

Theorem 2.4 (Conditional ELBO)**.**

$$\log p_\theta(\mathbf{x}|\mathbf{c}) \geq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x},\mathbf{c})} [\log p_\theta(\mathbf{x}|\mathbf{z}, \mathbf{c})] - D_{\mathrm{KL}} (q_\phi(\mathbf{z}|\mathbf{x}, \mathbf{c}) \| p(\mathbf{z}|\mathbf{c}))$$

Assuming $p(\mathbf{z}|\mathbf{c}) = p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ *(label-independent prior), the KL term is identical to the standard VAE.*

Label-Independent Prior

The assumption $p(\mathbf{z}|\mathbf{c}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ means that the prior distribution of "style" is the same regardless of the digit class. This is a simplification — in principle, one could learn class-dependent priors $p(\mathbf{z}|\mathbf{c})$, but the standard assumption works well in practice and simplifies training.

2.3.3 How Conditioning Changes the Architecture

The key architectural difference between a VAE and a CVAE is *where* and *how* the condition $\mathbf{c}$ is injected:

Conditioning Mechanisms

Encoder conditioning (spatial broadcast):

1. One-hot label $\mathbf{c} \in \{0, 1\}^{10}$ is reshaped to $(10, 1, 1)$
2. Broadcast to match image spatial dimensions: $(10, 28, 28)$
3. Concatenated with image along channel dimension: $(1, 28, 28) \oplus (10, 28, 28) = (11, 28, 28)$

This gives every spatial location access to the label information, allowing the convolutional layers to learn label-dependent features from the first layer.

Decoder conditioning (vector concatenation):

1. Sampled $\mathbf{z} \in \mathbb{R}^{32}$ is concatenated with $\mathbf{c} \in \mathbb{R}^{10}$
2. Combined vector $[\mathbf{z}; \mathbf{c}] \in \mathbb{R}^{42}$ is fed to the decoder

This is simpler than the encoder approach because the decoder starts from a vector (not a spatial feature map).

Alternative Conditioning Strategies

Other approaches exist:

- **Adaptive Instance Normalisation (AdaIN)**: condition modulates BN parameters (used in StyleGAN)
- **Cross-attention**: condition attends to features (used in Stable Diffusion)
- **FiLM**: condition produces scale and shift for feature maps

For this exercise, we use the simplest approach (concatenation) from the original CVAE paper (Sohn et al., 2015).

2.4 The β-VAE Framework

β-VAE (Higgins et al., 2017)

The generalised ELBO with a hyperparameter β:

$$\mathcal{L}_\beta = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x},\mathbf{c})}\left[\log p_\theta(\mathbf{x}|\mathbf{z},\mathbf{c})\right] - \beta \cdot D_{\mathrm{KL}}\left(q_\phi(\mathbf{z}|\mathbf{x},\mathbf{c}) \| p(\mathbf{z})\right)$$

- $\beta = 1$: standard VAE (ELBO)
- $\beta < 1$: weaker regularisation $\Rightarrow$ sharper reconstructions, less structured latent space
- $\beta > 1$: stronger regularisation $\Rightarrow$ blurrier output, more disentangled latent representations

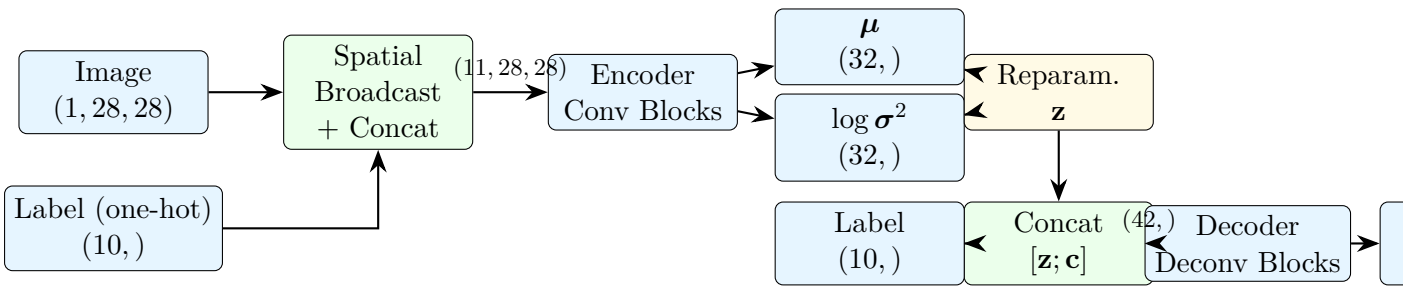
From an information-theoretic perspective, β controls the **rate-distortion trade-off**: higher β limits the information flow through the bottleneck (rate), forcing the model to rely more on the prior (which may increase distortion).

Common Misconception

Students often think $\beta > 1$ is always better because "more regularisation = more structure". In practice, excessively high β causes **posterior collapse**: the encoder learns to ignore the input entirely, outputting $q(\mathbf{z}|\mathbf{x}) \approx p(\mathbf{z})$ for all $\mathbf{x}$. The decoder then ignores $\mathbf{z}$ and relies solely on the label, producing low-diversity outputs.

3 Architecture Details

3.1 Overview



3.2 Encoder Architecture

Layer	Type	Config	Output Shape	Notes
Input	—	—	$(B, 11, 28, 28)$	Image + broa
Conv Block 1	Conv2d + BN + LeakyReLU	$k=3,\ s=2,\ p=1,\ C=32$	$(B, 32, 14, 14)$	Stride 2 down
Conv Block 2	Conv2d + BN + LeakyReLU	$k=3,\ s=2,\ p=1,\ C=64$	$(B, 64, 7, 7)$	Stride 2 down
Conv Block 3	Conv2d + BN + LeakyReLU	$k=3,\ s=1,\ p=1,\ C=128$	$(B, 128, 7, 7)$	No downsamp
Flatten	—	—	$(B, 6272)$	$128 \times 7 \times 7$
`fc_mu`	Linear	$6272 \to 32$	$(B, 32)$	Mean of $q(\mathbf{z}\|\mathbf{x}$
`fc_log_var`	Linear	$6272 \to 32$	$(B, 32)$	Log-variance

3.3 Decoder Architecture

Layer	Type	Config	Output Shape	Notes
Input	—	—	$(B, 42)$	$\mathbf{z}$ (32) + la
`decoder_fc`	Linear + LeakyReLU	$42 \to 6272$	$(B, 6272)$	Expand to
Reshape	—	—	$(B, 128, 7, 7)$	Ready for
Deconv Block 1	ConvT2d + BN + LeakyReLU	$k=3,\ s=1,\ p=1,\ C=64$	$(B, 64, 7, 7)$	No upsamp
Deconv Block 2	ConvT2d + BN + LeakyReLU	$k=4,\ s=2,\ p=1,\ C=32$	$(B, 32, 14, 14)$	Stride 2 up
Deconv Block 3	ConvT2d + Sigmoid	$k=4,\ s=2,\ p=1,\ C=1$	$(B, 1, 28, 28)$	Stride 2 up

Kernel Size 4 for Upsampling

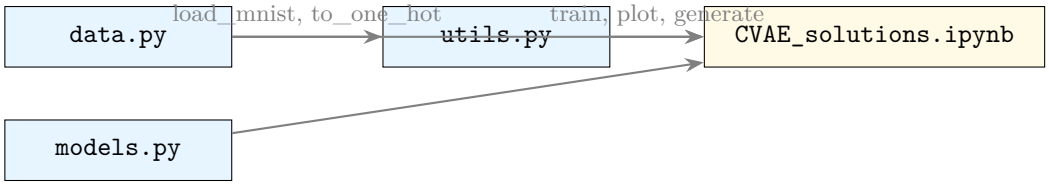
We use $k = 4$ with $s = 2$, $p = 1$ for upsampling layers. This ensures exact doubling of spatial dimensions: $H_{\text{out}} = (H_{\text{in}} - 1) \times s - 2p + k = (7 - 1) \times 2 - 2 + 4 = 14$. Using $k=3$ with $s=2$ would require `output_padding=1` to achieve the same result.

3.4 Design Decisions: MNIST vs. CelebA

Aspect	Original (CelebA)	Exercise (MNIST)
Image size	$64 \times 64 \times 3$ (RGB)	$28 \times 28 \times 1$ (Grayscale)
Condition	40 binary attributes	10-class one-hot
Latent dim	128	32
Encoder depth	4 conv blocks (stride 2 each)	3 conv blocks (2 with stride 2)
Decoder depth	5 deconv blocks	3 deconv blocks
Framework	TensorFlow 2.x	PyTorch
Training time	Hours (full CelebA)	Minutes (MNIST)

The core concepts (conditioning, reparametrization, ELBO, β trade-off) are **identical** — only the scale differs.

4 File Relationships



***data.py**: Data loading only*
***models.py**: Classifier for evaluation*

***utils.py**: All training loops, plotting, generation, and evaluation helpers*

***Notebook**: ConvCVAE class,*
loss function (student TODOs),
all pedagogical content

4.1 Why the Model Lives in the Notebook

Following the Autoencoders CX pattern, the `ConvCVAE` class and `cvae_loss` function are defined *in the notebook* rather than in a support file. This ensures:

- Students see and modify the model directly
- The TODO structure is clear and self-contained
- No need to switch between files during the exercise

4.2 Function Reference

File	Function	Description
`data.py`	`load_mnist()`	Downloads MNIST, returns numpy arrays normalised to $[0, 1]$
	`to_one_hot(y, n)`	Converts integer labels to one-hot float32
`models.py`	`MNISTClassifier`	CNN classifier for evaluating generation quality
`utils.py`	`setup()`	Confirms device (CPU/GPU)
	`train_classifier()`	Trains MNISTClassifier on labelled data
	`train_cvae()`	Main CVAE training loop (Adam + cosine schedule)
	`plot_losses()`	Plots total/recon/KL loss curves
	`reconstruct_cvae()`	Forward pass, returns numpy reconstructions
	`plot_original_vs_reconstructed()`	Side-by-side comparison plot
	`plot_per_digit_mse()`	Per-digit reconstruction error analysis
	`generate_conditional()`	Sample $\mathbf{z}$ + label $\rightarrow$ generate and display
	`generate_digit_grid()`	10×N grid of generated digits
	`interpolate_latent()`	Latent space interpolation with optional label switch
	`visualise_latent_space_cvae()`	PCA scatter of $\boldsymbol{\mu}$ coloured by digit
	`evaluate_and_plot_confusion()`	Classifier accuracy + confusion matrix
	`evaluate_generated_images()`	Generate per-digit, classify, report accuracy
`notebook`	`ConvCVAE`	The CVAE model class (student TODOs)
	`cvae_loss()`	ELBO loss function (student TODO)

5 TODO Solutions and Grading Notes

5.1 TODO 1: ConvCVAE Methods

TODO 1a: `conditional_input`

```
def conditional_input(self, x, label):
    label_map = label.unsqueeze(-1).unsqueeze(-1)
    label_map = label_map.expand(-1, -1, x.size(2), x.size(3))
    return torch.cat([x, label_map], dim=1)
```

Key point: Students must understand broadcasting. Common error: trying to use `label.view(B, 10, 28, 28)` directly (wrong — that reshapes rather than broadcasts).

TODO 1b: `reparametrize`

```
def reparametrize(self, mu, log_var):
    std = torch.exp(0.5 * log_var)
    eps = torch.randn_like(std)
    return mu + std * eps
```

Key point: Must use `log_var * 0.5` (not `log_var ** 0.5`). The encoder outputs *log-variance*, not variance.

TODO 1c–e: `encode`, `decode`, `forward`

These are straightforward sequential compositions. Common errors:

- Forgetting `.flatten(1)` between conv and fc layers
- Wrong reshape dimensions in `decode` (must be $128 \times 7 \times 7$)
- Forgetting to concatenate label in `decode`

5.2 TODO 2: Loss Function

TODO 2a–b: BCE + KL

```
recon_loss = F.binary_cross_entropy(recon_x, x, reduction='sum') / batch_size
kl_loss = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp()) / batch_size
```

Key point: Students may confuse `reduction='sum'` vs `'mean'`. We sum over pixels then average over batch for consistency with the KL term.

5.3 TODOs 3–5: Open-Ended Exploration

These are not graded for correctness but for engagement. Look for:

- TODO 3: Did they try multiple digits? Did they notice style variation?
- TODO 4: Did they compare fixed-label vs. label-switch interpolation?
- TODO 5: Did they articulate the β trade-off?

6 Expected Student Difficulties

Difficulty 1: "Why is the VAE blurrier than the AE?"

The KL term prevents the encoder from using the full capacity of the latent space. This is by design — it creates a smooth latent space that enables generation. The AE has no such constraint, so it can achieve lower reconstruction error at the cost of a "holey" latent space.

Difficulty 2: Shape Errors in `conditional_input`

The spatial broadcast of labels is conceptually simple but easy to get wrong. Students may try `label.repeat(1, 1, 28, 28)` (wrong — repeat duplicates along existing dims, doesn't add new ones). The correct pattern is `unsqueeze` then `expand`.

Difficulty 3: "Why does the latent space overlap more?"

Students who completed the AE exercise expect clear digit clusters. In a CVAE, the label carries identity, so $\mathbf{z}$ captures shared style features. More overlap = better disentanglement. Frame this as a *feature*, not a bug.

Difficulty 4: KL Loss Going Up

Students may be alarmed that KL loss increases during training. Explain: at initialisation, the encoder outputs near-zero μ and $\log \sigma^2$, so KL ≈ 0. As the encoder learns to encode information in $\mathbf{z}$, KL increases. It should eventually plateau.

7 Discussion Points for Class

1. **AE vs. CVAE latent spaces:** Why does the CVAE latent space show more overlap between digit classes? Is this desirable?
2. **Continuous conditions:** What if instead of a digit label, we conditioned on a continuous variable (e.g., rotation angle)? How would the architecture change?
3. **Connection to diffusion models:** Modern text-to-image models (Stable Diffusion, DALL-E) also use conditioning. How is their approach similar to / different from a CVAE?
4. **The CelebA version:** The original project used 40 binary face attributes. What new capabilities does this enable? (Attribute manipulation: "add glasses", "change hair colour" while preserving identity.)
5. **Posterior collapse:** What happens with very high β? The encoder may learn to ignore the input entirely ("posterior collapse"). How would you detect this?
6. **Label-dependent priors:** We assumed $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ for all labels. What if different digits have different priors? Would this help or hurt?

8 References

1. Kingma, D.P. & Welling, M. (2014). *Auto-Encoding Variational Bayes*. ICLR.
2. Sohn, K., Lee, H., & Yan, X. (2015). *Learning Structured Output Representation using Deep Conditional Generative Models*. NeurIPS.

3. Higgins, I. et al. (2017). *β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework*. ICLR.

4. Doersch, C. (2016). *Tutorial on Variational Autoencoders*. arXiv:1606.05908.